

SOLUCIONS A L'EXAMEN FINAL de 1a CONVOCATÒRIA 2022-23

A continuació us ofereixo les solucions a l'examen, de manera que els estudiants que hagin suspès en primera convocatòria puguin veure on s'havien equivocat i com s'havia de resoldre el problema. Generalment prefereixo no compartir les solucions ja que els programes es poden fer de mil maneres diferents, i la que jo us ofereixo és només una. Per tant, que la vostra solució no coincideixi exactament amb la que aquí us proposo no vol dir que la vostra opció no estigui bé. Les solucions que us proposo sempre intentaran ser les més senzilles, sense sobrecomplicacions.

EXERCICI 1. [2.5 PUNTS]. Formar paraula: **SOLUCIÓ**

Especificació del procediment (aquesta especificació no es pot canviar):

- Paràmetres d'entrada:
 - o seq: taula[] de caràcter (que conté la seqüència de lletres)
 - o par: taula[] de caràcter (que conté la paraula buscada)
- Valor de retorn: booleà.

Suposarem que tant la cadena "seq" com la cadena "par" estan en majúscules. No cal comprovar-ho.

Per resoldre aquest exercici necessitem veure si amb les lletres que tenim a "seq" podem formar la paraula "par". No es permet reutilitzar lletres, de manera que si jo he de fer "PATATA" i només tinc la seqüència "TAP", la resposta ha de ser que no, perquè no puc fer servir la "T" dues vegades ni la "A" tres vegades.

Per tant, és crucial mantenir algun tipus de compteig que faci que no reutilitzem lletres. Hi ha moltes maneres de fer-ho. La més fàcil, possiblement, és sobreesciure la seqüència "seq" amb un caràcter diferent de l'alfabet de tal manera que ja sabem que l'hem fet servir. Alguns de vosaltres l'heu resolt posant-ho a minúscules, per exemple, que és una bona solució per no perdre la informació de la seqüència original. Si ens és igual perdre aquesta informació, podem sobreesciure-ho amb qualsevol caràcter, (e.g. "*"), i si necessitem mantenir la informació original sempre podem fer una còpia de l'string inicial i treballar sobre la còpia (això requereix memòria dinàmica per fer-ho bé). En aquesta pràctica us deixava que sobreescrivissiu la seqüència sense penalització. Una altra manera seria crear una taula de booleans (o enters), de la mateixa mida que la seqüència original (per tant, amb memòria dinàmica si volem ser exactes) inicialitzat a fals (o zero), sent que aquesta taula indica si s'ha fet servir o no la lletra que coincideix amb aquella posició. Hi ha d'altres maneres de fer-ho, per exemple també podem, per cada lletra de la paraula original, comptar quantes en necessitem, i després recórrer la seqüència i comptar quantes lletres com la que necessitem hi ha.

A continuació us poso la solució més senzilla, que és la que sobreesciu la seqüència original.

funció formar_paraula (seq: taula[] de caràcter, par: taula[] de caràcter) **retorna** booleà
és

```
const
    FINAL := '\0';
fconst
var
    i, j: enter;
    lletres: enter;
    mida_par : enter;
    ll_trobada: booleà;
fvar
inici
    $ Compto la mida de la paraula
    mida_par := 0;
    mentre (par[mida_par] ≠ FINAL) fer
        mida_par := mida_par + 1;
    fmentre

    i := 0;
    lletres := 0;
    $ Recórrer cada lletra de la paraula que volem formar
    mentre (par[i] ≠ FINAL) fer
        j := 0;
        ll_trobada := fals;
```

```

mentre (seq[j] ≠ FINAL) i no(ll_trobada) fer
  si (par[i] = seq[j]) llavors
    $ Si el trobo és que el puc fer servir
    $ Sobreescric amb minúscula, però podria ser qualsevol cosa.
    seq[j] := seq[j] - 'A' + 'a';
    lletres := lletres + 1; $ Comptador de lletres trobades
    ll_trobada := cert;
  fsi
  j := j + 1;
fmentre
i := i + 1;
fmentre
si (lletres = mida_par) llavors
  retorna (cert);
sinó
  retorna (fals);
fsi
ffunció

```

EXERCICI 2 [2.5 PUNTS]. El set i mig. SOLUCIÓ

Aquest exercici era molt fàcil de solucionar. Alguns us heu sobre-complicat guardant els nombres en una taula. O bé heu considerat que la funció `llegir_f()` és capaç de llegir taules d'enters de cop, cosa que no pot fer. Alguns heu llegit la seqüència sencera com si fos una cadena, cosa que sí que pot fer la funció, però llavors teníeu una taula de caràcters que havíeu de transformar a numèric, anar en compte amb els espais, i anar en compte amb els nombres de dues xifres: fer-ho així era complicar-se la vida a més no poder. De la mateixa manera, guardar-ho en una taula requeria fer-ho amb memòria dinàmica, i era totalment innecessari ja que tots els càlculs es poden fer d'una sola passada, mentre llegim el fitxer.

El codi a continuació obre el fitxer, fa les comprovacions pertinents, i llegeix nombre a nombre. Cada nombre el tradueix per una puntuació i n'acumula el resultat. S'ha d'anar en compte que la variable on guardem la puntuació ha de ser tipus real, perquè ha de poder guardar els 0.5.

```
algorisme set_i_mig és
const
    SENTINELLA := -1;
    NOM_FITXER := "partides.txt";
fconst
var
    f_in: fitxer;
    num_llegit: enter;
    num_partida: enter;
    num_passades, num_guanyades, num_perdudes: enter;
    punts: real;
fvar
inici
    $ Inicialitzar variables
    punts := 0.0;
    num_partida := 1;
    num_passades := 0;
    num_perdudes := 0;
    num_guanyades := 0;

    $ Obrir fitxer
    f_in := obrir_f (NOM_FITXER, "l");
    $ Comprovar si existeix
    si (f_in = NUL) llavors
        error("Fitxer no existeix");
    fsi

    $ Llegir primera carta (ens diuen que fitxer correcte i conté partides)
    llegir_f (f_in, num_llegit);
    mentre (no (final_f(f_in))) fer
        $ Tractar la subseqüència
        punts := 0.0;
        mentre (num_llegit ≠ SENTINELLA) fer
            $ tractar carta
            si (num_llegit >= 10 i num_llegit <=12) llavors
                punts := punts + 0.5;
            sinó
                punts := punts + num_llegit;
            fsi
            $ llegir següent
            llegir_f(f_in, num_llegit);
        fmentre
        $ Hem acabat amb la subseqüència: hem acabat una partida
        escriure ("La partida", num_partida, "té un total de", punts, "punts.");
        si (punts < 7.5) llavors
            escriure(" El jugador ha passat.");
            num_passades := num_passades + 1;
        sino si (punts = 7.5) llavors
            escriure(" El jugador ha guanyat.");
            num_guanyades := num_guanyades + 1;
        sino
            escriure(" El jugador ha perdut.");
            num_perdudes := num_perdudes + 1;
```

```

    fsi
    num_partida := num_partida + 1;
    $ Següent subseqüència
    llegir_f(f_in, num_llegit);
fmentre
$ Imprimir resultats totals
escriure("Nombre de partides totals =", num_passades+num_guanyades+num_perdudes);
escriure("Nombre de partides passades = ", num_passades);
escriure("Nombre de partides guanyades = ", num_guanyades);
escriure("Nombre de partides perdudes = ", num_perdudes);
tancar_f(fitxer);
falgorisme

```

EXERCICI 3. [2.5 PUNTS]. Procediment que implementa atoi(). [SOLUCIÓ](#).

Especificació del procediment:

- Paràmetres d'entrada:
 - o s: taula[] de caràcter (que conté el nombre en format string)
- Valor de retorn: enter.

En aquest exercici havíem de fer un procediment que convertís una cadena de caràcters que conté dígitos al seu valor numèric. Per exemple, si la cadena era “1234”, havia de retornar el valor numèric 1234 (mil dos-cents trenta-quatre).

Per fer-ho és senzill si recordem que en el sistema decimal, la posició de cadascuna de les xifres (començant per la dreta i per zero) és l'exponent al qual hem d'eleva la base 10. Tenim disponible una funció anomenada “potència” que calculava la potència, tot i que alguns heu preferit anar multiplicant per 10 dins del bucle (també correcte). El que no és correcte és fer servir com a exponent una constant (e.g. longitud), llavors no canvia de valor a cada bucle i no funciona. D'altres us heu equivocat per una unitat i el nombre generat, en comptes de ser 1234, era 12340.

```
funció atoi (s: taula[] de caràcter) retorna enter és
var
    num : enter;
    len : enter;
    i   : enter;
fvar
inici
    num := 0;
    len := 0;
    mentre (s[len] ≠ '\0') fer
        len := len + 1;
    fmentre

    per ( i:=0; i<len; i:=i+1) fer
        $ També funcionaria:
        $ per ( i:=len-1; i>=0; i:=i-1) fer
            num := num + (s[i]-'\0') * potencia(10, len-i-1);
    fper
    retorna(num);
ffunció
```

EXERCICI 4. [2.5 PUNTS]. La funció misteriosa. SOLUCIÓ.

```
funció funció_misteriosa (a: taula[] d'enter, b: enter) retorna enter és
var
    p, q, r: enter;
fvar
inici
    p := 0;
    $ La funció llargada retorna el nombre d'elements
      d'un vector d'enters
    q := llargada(a) - 1;

fer
    r := (p + q) div 2;
    si (b <= a[r]) llavors
        q := r - 1;
    fsi
    si (a[r] <= b) llavors
        p := r + 1;
    fsi
mentre (p <= q);

si (a[r] = b) llavors
    retorna r;
sinó
    retorna -1;
fsi
ffunció
```

Considereu la funció següent.

a) Què fa la funció següent?

Aquesta funció implementa l'algorisme de cerca dicotòmica / binària, el qual requereix que el vector estigui ordenat creixentment per funcionar. L'algorisme cercarà l'enter "b" dins la taula "a". Retornarà la posició on es troba l'element si l'hem trobat o -1 si no l'hem trobat.

b) Què retorna si $a=\{1,2,3,4,5\}$ i $b=2$?

Retorna 1, que és la posició on es troba l'element 2.

c) Què retorna si $a=\{23, 45, 54, 65, 99\}$ i $b=50$?

Retorna -1, ja que l'element 50 no hi és.

d) Què retorna si $a=\{13, 27, 42, 85, 99, 63, 34\}$ i $b=63$?

Retorna -1, ja que tot i ser-hi, no el troba perquè no està ordenat.

e) Quin és el cost algorísmic d'aquest procediment?

El cost d'aquest algorisme és logarítmic respecte la mida del vector "a". Per tant, si la mida és n, seria $\log_2(n)$. O bé aquest concepte se sap de teoria, o bé per veure-ho fent-ho manualment, es veu que hi ha un únic bucle que es fa mentre p sigui menor o igual a q. Per tant la pregunta és, quantes vegades es fa aquest bucle? Quan p valdrà el mateix que q? Com que "p" inicialment val 0 i q inicialment val mida-1, estan cadascun en un extrem. Imaginem que estem buscant un nombre que està a la primera posició. Com que a cada iteració dividim el vector en dues parts, a la segona iteració p seguirà a la posició inicial i q valdrà la mida dividit de 2. A la següent iteració p es queda igual i q valdrà el que valia dividit de 2. Cada cop estem dividint per 2, cosa que ens dona un cost logarítmic.